Preface

This slide deck is from W25 on the topic of Dynamic Memory. We are providing it unmodified for W26 for those of you who prefer slides.

You do not need to know anything about linked lists for W26, though the example is still very likely to be helpful!

You also do not need to know about function pointers, but you may enjoy the example in that section as well.

Slides – Vybihal & Bérubé-Vallières © 2023

Lecture 18

Dynamic Memory

Readings: textbook Section 3 Example 8

Slides – Vybihal & Bérubé-Vallières © 2023

Dynamic Storage Allocation

- Up to now, we have seen that data structures in C usually have fixed size determined at compile time.
  - int array[10]; // At compile time
- We needed to edit the program and compile it again if we wanted to use a different size
- C provides functions to allocate memory dynamically instead (i.e. during runtime)

Slides – Vybihal & Bérubé-Vallières © 2023

C's dynamic memory functions

#include<stdlib.h>

Remember that you can think of size_t as an integer

```
void *malloc(size_t size);
```

- Allocates a block of size `size` in memory and returns a pointer to it
- Does not initialize it
- This is the most used memory allocation function

```
void *calloc(size_t num, size_t size);
```

- Allocates a block of size `size * num` in memory and return a pointer to it (i.e. space for an array of `num` elements that have size `size`)
- Initializes it by setting all bits to 0

Both malloc and calloc return a null pointer (NULL) if not able to find large enough memory space.

```
void free(void *);
```

- Releases the block of memory pointed by the pointer (needs to have been previously assigned by a memory allocation function, not just any pointer)
- As long as you don't do this, the memory is not released. Anytime you malloc, you will need to release it later. C doesn't have automatic garbage collection

Since malloc and calloc don't know the type of data you are going to store in memory, they return a void pointer, which can be seen as a generic pointer.

- This needs to be casted (implicitly or explicitly) into the proper pointer type when used.

Example (Allocating storage for uninitialized array)

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    int *array;
    int n;
    scanf("%d", &n);
    array = malloc(n*sizeof(int)); // even if you know that sizeof(int) is 4 on
                                   // your machine, it I still good programming habit to stick with sizeof
    if (array == NULL) exit(1);
    array[0] = 5;
    printf("%d", array[0]);
    free(array);

    return 0;
}
```

Pointer

Implicit type cast from void* to int*. Sometimes you will see (int*) malloc(…)
People argue about which is better:
https://en.wikipedia.org/wiki/C_dynamic_memory_allocation#Advantages_to_casting

The amount of memory you want to allocate

Null pointer

What happens if you don't free before the end? In a modern OS, the OS will reclaim the memory. However, if this is used in another environment, the data might not be released. Good practice is to free memory as soon as you don't need it, otherwise your program may run out of memory. Every time that malloc/calloc is called, free should be called later.

Example (Allocating storage for 0-initialized array)

```
#include <stdio.h>
#include <stdlib.h>
 int main(void) {
     int *array;
     int n;
     scanf("%d", &n);
     array = calloc(n, sizeof(int)); // compare to malloc(n*sizeof(int));

     if (array == NULL) exit(1);

     array[0] = 5;
     printf("%d", array[0]);
     free(array);

     return 0;
}
```

Example (using malloc to return a new array/string from a function)

Last time, we saw that if we wanted to return a new array/string from a function, we either had to give it a buffer array for the function to fill with the array/string, or declare it statically, since otherwise the function releases the memory once it ends. Another way is to allocate the memory ourselves:

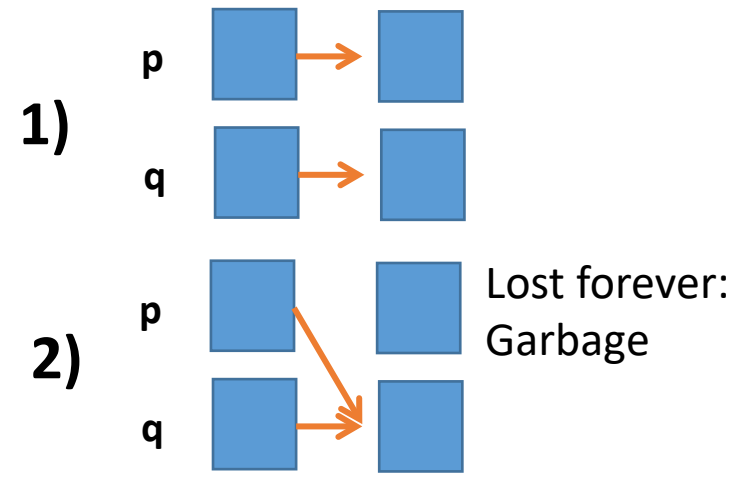
```
#include <stdio.h>
#include <stdlib.h>
char *get_string(int size){
    printf("Enter a string of at most size %d:\n", size);
    char *str = malloc ( size + 1 ); // +1 for null termin.
         // Note: sizeof(char) is guaranteed to be 1, no need to do (size+1)*sizeof(char)
    fgets(str, size, stdin);
    return str;
}
int main(void) {
   char *input = get_string(50);
   printf("%s", input);
   free(input); // Must be freed!
}
```


Slides – Vybihal & Bérubé-Vallières © 2023

Deallocation, garbage, memory leaks

- Calling too many memory allocation functions without releasing the blocks of memory afterwards may exhaust the pool of memory they have access to (the heap) and cause the functions to return null pointers. For each call to malloc, free must later be called.
- It is also possible to forever lose track of some memory that is still allocated. For example:

```
int *p = malloc(sizeof(int));
int *q = malloc(sizeof(int));
p = q;
```



- Memory that is allocated but has no references to it is called garbage
- A program that leaves garbage behind has a memory leak.
- Many programming languages offer a garbage collector that automatically find and release garbage. C does not.

Dangling pointers

When using free, although the memory is freed, the pointer still exists and points to it. This is called a dangling pointer. Be careful to not use that pointer again!

```
int *p = malloc(sizeof(int));

[…]

free(p);

[…]

*p = 4; // Undefined behaviour, this location in memory has been
        // freed!
```


Slides – Vybihal & Bérubé-Vallières © 2023

What about VLAs?

(You do not need to know about VLAs, and you should use fixed size or malloc instead in most cases: this slide is for people who know about them)

- Some of you might have learned or discovered by yourself that this works:

```
int n=5;
int array[n];
// or even
int n;
scanf("%d", &n);
int array[n];
```

- This is a C99 feature called Variable Length Arrays, which lets you fix the length of an array at runtime. However, the feature was not well-received.
  - C11 made this feature optional (compilers don't have to implement it)
  - C23 made VLA types mandatory again as arguments to functions, but they are still optional as local variables.
- It has a couple problems: One is that it (usually) allocates memory on the stack, which is small compared to the heap (could be around 8 MB)
  - If not enough space: Unlike malloc, nothing like a NULL pointer, your program will just segfault
- So, for the two use cases above, the first one is not very useful since you know the size and could just make a regular array, and the second one is unsafe. You should use fixed size or malloc.

Slides – Vybihal & Bérubé-Vallières © 2023

Example 1 (with pointer to struct)

```
struct STUDENT {
    int age;
    float GPA;
};

struct STUDENT *x;

x = malloc(sizeof(struct STUDENT));
if (x == NULL) exit(1);

// First way to access the content:
(*x).age = 5;        // Form 1
// However, this is such a common operation that there is a shortcut called right
    arrow selection:
x->age = 5;          // Form 2 (This is equivalent to form 1)
```

Careful, *x.age would mean *(x.age), i.e dereference the age member

Slides – Vybihal & Bérubé-Vallières © 2023

Example 2 (with pointer to struct)

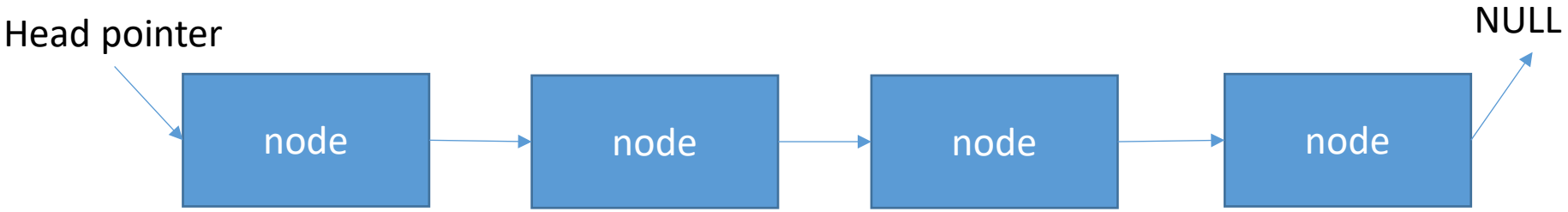
```
struct STUDENT *students, *aStudent; // using same def as previous slide
int n, x;

scanf("%d", &n);
students = calloc(n, sizeof(struct STUDENT)); // an array of STUDENTs

if (students == NULL) exit (1);

for(x=0; x<n, x++)  {
    aStudent = students+x;
    scanf("%d %f", &(aStudent->age), &(aStudent->GPA));
}

// or: &((students+x)->age)
```


Slides – Vybihal & Bérubé-Vallières © 2023

Linked Lists

Dynamic storage allocation is very useful when making more complex data structures.

Once such structure is the linked list. Up to now, we have been making arrays when we wanted a collection of items. Linked lists are an alternative that lets use insert and delete nodes, which can make the list grow or shrink in size gradually, as needed. They look like this:



Slides – Vybihal & Bérubé-Vallières © 2023

Node

This is what a simple linked list node would look like

When a structure refers to itself as a member, a tag is needed, typedef would not work! (Although you could still have both the typedef and the tag)

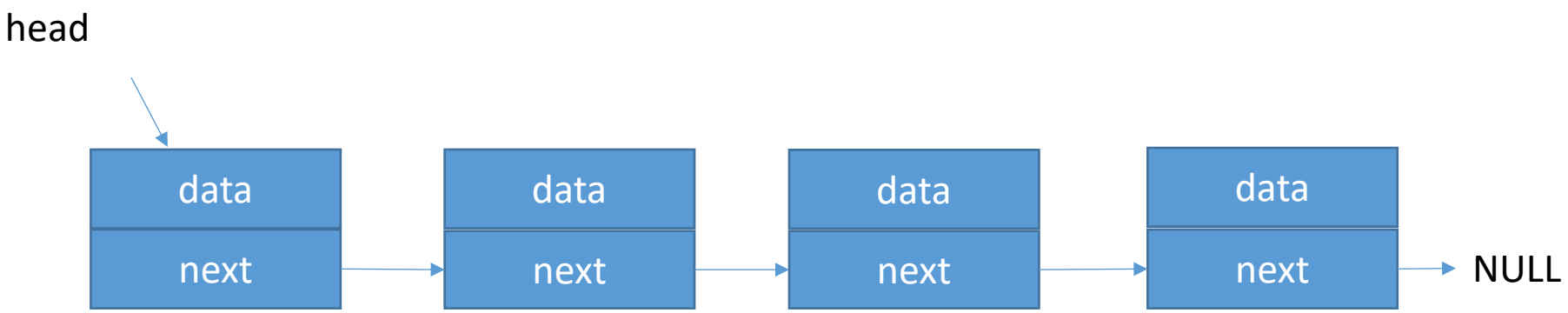
```
struct NODE {
   int data;            // the data we want to store in the node
   struct NODE *next;   // the pointer to the following node
                        // in the list
 };
```

To keep track of where the list begins, we need a variable that always points the first node (the head). As we currently have no node, we can first set it to NULL to indicate the linked list is empty

```
struct NODE *head = NULL;
```


Slides – Vybihal & Bérubé-Vallières © 2023

Example of Linked List



Creating a node

To create a node, we need to do the following:

1. Allocate memory for the node
2. Store data in the node
3. Insert the node in the list

Previous slide:

```
struct NODE {
        int data;
        struct NODE *next;
}
[…]
struct NODE *head = NULL;
```

```
struct NODE *aNode;  // variable that will temporarily point
                     // to the node until we insert it

 aNode = malloc(sizeof(struct NODE));  // 1. Allocate memory
                                       // for the node

 if (aNode == NULL) exit(1);

 aNode->data = 5; // 2. store data [remember this is
                  // equivalent to(*aNode).data = 5;]

// 3. Inserting the node is on next slide
```

Inserting a node to beginning of list

To insert a node in a linked list we need to:

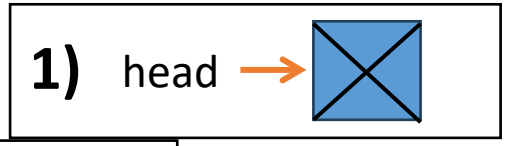
1. Make the "next" member of the node point to the current head
2. Make the head variable point to the node

```
aNode -> next = head;
head = aNode;
```

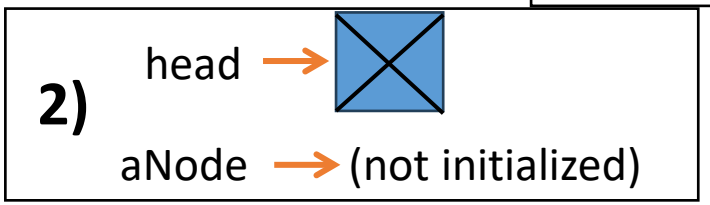

Slides – Vybihal & Bérubé-Vallières © 2023

Previous examples together

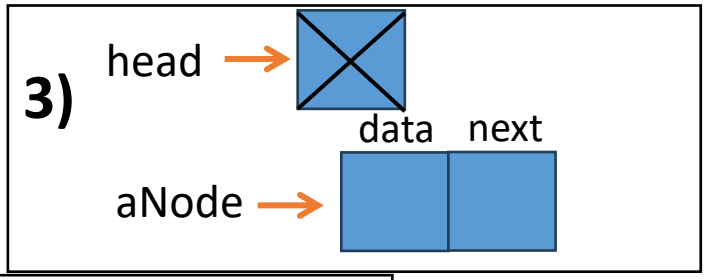
```
struct NODE {int data; struct NODE *next;};
struct NODE *head = NULL;
```



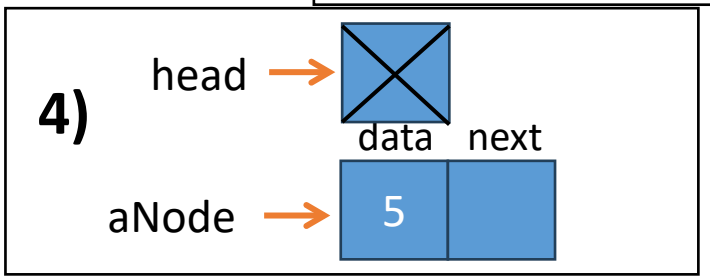

```
struct NODE *aNode;
```



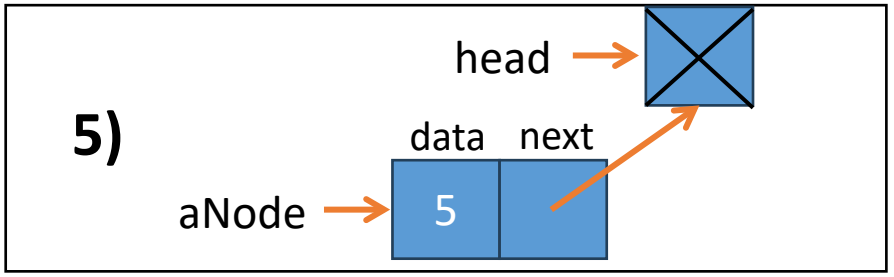

```
aNode = malloc(sizeof(struct NODE));
if (aNode == NULL) exit(1);
```



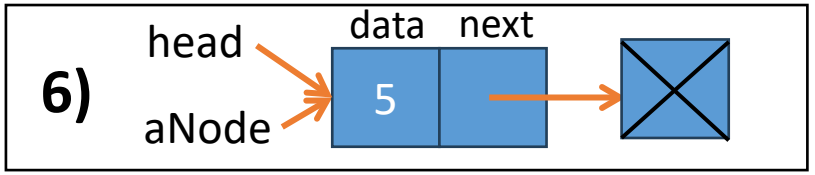

```
aNode->data = 5;
```



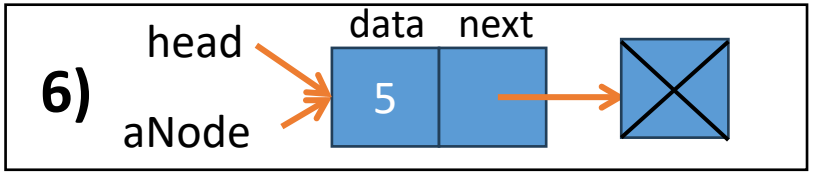

```
aNode -> next = head;
```



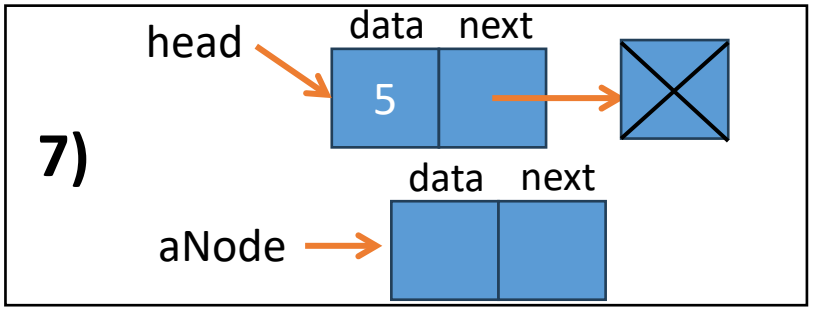

```
head = aNode;
```



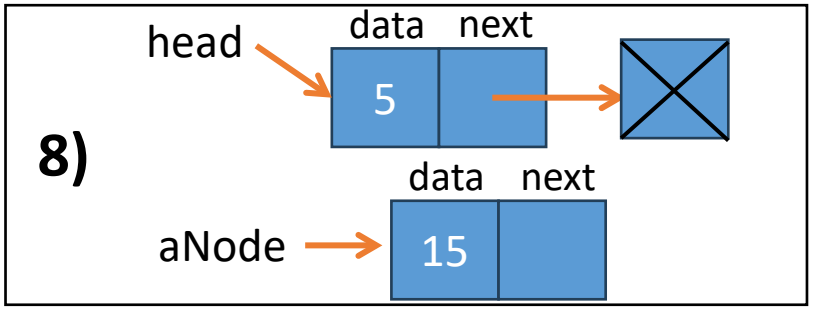
Adding one more



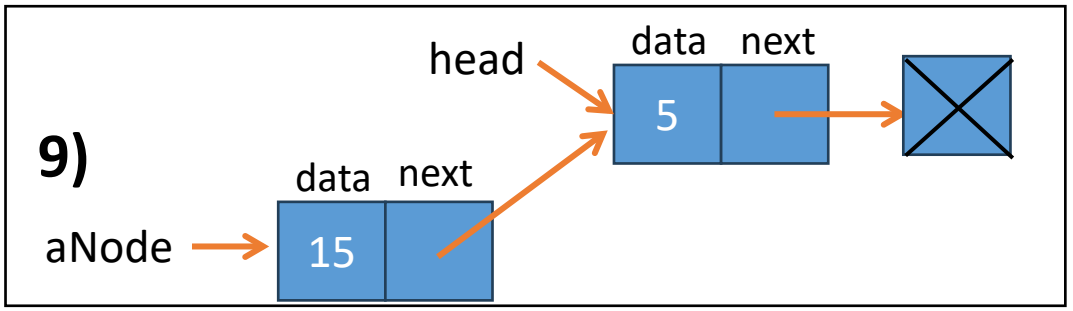

```
aNode = malloc(sizeof(struct NODE));
```



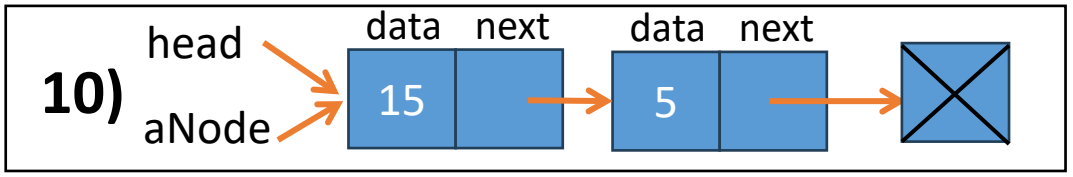

```
aNode->value = 15;
```




```
aNode->next = head;
```




```
head = aNode;
```



Function to insert a node to front of list

- This adds a node with `someData` in front of node `ptr` (Note: This doesn't edit the list itself, it returns the new node (which will now be the head), which then needs to be stored in the head pointer.

```
struct NODE* addNode(struct NODE *ptr, int someData) {
        struct NODE *temp = malloc(sizeof(struct NODE));
        if (temp == NULL) return NULL;     // NULL to designate error
        temp->data = someData;
        temp->next = ptr;        // We add it in front of ptr
        return temp;                     // return the new node;
}
[…]
```

// The equivalent to last slides would then be

```
struct NODE *head = NULL;
head = addNode(head, 5);
head = addNode(head, 15);
```

Other example using addNode

```
int main() {
    struct NODE * head = NULL;
    int x, newData;
    for(x=0; x<10; x++)  {
        scanf("%d", &newData);
        head = addNode(head, newData);
 }
```


Slides – Vybihal & Bérubé-Vallières © 2023

Printing all elements

```
void printNodes(struct NODE *ptr) {          // copy of head pointer
    while (ptr != NULL) {                    // stop at end of list
        printf("%d\n", ptr->data);
        ptr = ptr->next;                     // move to the following node in list
    }
}

int main() {
    struct NODE * head = ... A list previously created…
    printNodes(head);
}
```

Question

How would we define the structure that stores students in a linked list?

- Assume student has:
  - Name
  - Age
  - GPA

Slides – Vybihal & Bérubé-Vallières © 2023

Pointer to pointer (1/2)

- Remember addNode…

```
struct NODE* addNode(struct NODE *ptr, int someData) {
        struct NODE *temp = malloc(sizeof(struct NODE));
        if (temp == NULL) return NULL;     // NULL to designate
   error
        temp->data = someData;
        temp->next = ptr;        // We add it in front of ptr
        return temp;                     // return the new node;
}
[…]
struct NODE *head = NULL;
head = addNode(head, 5);
head = addNode(head, 15);
```

- What if we wanted to edit the head pointer directly from the function instead? We would give a pointer to it! (Next slide…)

Pointer to pointer (2/2)

```
void addNode2(struct NODE **ptr, int someData) {
        struct NODE *temp = malloc(sizeof(struct NODE));
        if (temp == NULL) return NULL;     // NULL to designate
   error
        temp->data = someData;
        temp->next = *ptr; // we dereference to get the
                           // head pointer
        *ptr = temp;
}
[…]
struct NODE *head = NULL;
addNode2(&head, 5); // we give the pointer to head pointer
addNode2(&head, 15);
```


Slides – Vybihal & Bérubé-Vallières © 2023

Pointers to functions (1/3)

Functions also have addresses, and you can point to them.

While you probably won't need this for COMP 206, this is surprisingly useful, and you WILL see it in the future. C's sorting functions use it, and you'll probably see it in COMP 310.

```
int sum(int a, int b) {
   return a+b;
}
void hello_world() {
   puts("Hello World!");
}
int main(void) {
   int (*sumptr)(int, int); // Declaring a pointer to a function that
                            // takes two ints as argument and
                                    // returns an int
   void (*helloptr)(); // Pointer to a function that takes no
                                 // argument and returns nothing
   sumptr = &sum;
   helloptr = &hello_world;
   int x = (*sumptr)(10, 5);
   (*helloptr)();
}
```

Need the parentheses around *sumptr since otherwise that would attempt to dereference the return value of sum!

Pointers to functions (2/3)

Functions also have addresses, and you can point to them.

```
int sum(int a, int b) {
   return a+b;
}
void hello_world() {
   puts("Hello World!");
}
int main(void) {
   int (*sumptr)(int, int); // Declaring a pointer to a function that
                            // takes two ints as argument and
                                    // returns an int
   void (*helloptr)(); // Pointer to a function that takes no
                       // argument and returns nothing
   sumptr = sum;
   helloptr = hello_world;
   int x = sumptr(10, 5);
   helloptr();
}
```

Similar to what we saw with arrays:
-The compiler understands that using a function name is equivalent to its pointer
-The compiler understands that a pointer followed by () is a function call
This code is equivalent to last slide

Pointers to functions (3/3)

One of the many uses is that functions can be used as parameters

```
void sorted(int* arr, int len, int (*compare)(int, int)) {
    int swap;
    do { swap = 0;
         for (int i = 1; i < len; i++) {
         if (compare(arr[i-1], arr[i]) < 0 ) {
              int temp = arr[i-1];
              arr[i-1] = arr[i];
              arr[i] = temp;
              swap = 1;
    }}} while (swap);
}
int increasing(int a, int b) { return b-a; } // returns > 0 if b > a;
int decreasing(int a, int b) { return a-b; } // returns > 0 if a > b;
int main(void) {
    int arr[]={5,2,4,3,5};
    sorted(arr, 5, decreasing); // arr is now {5,5,4,3,2}
    sorted(arr, 5, increasing); // arr is now {2,3,4,5,5}
}
```